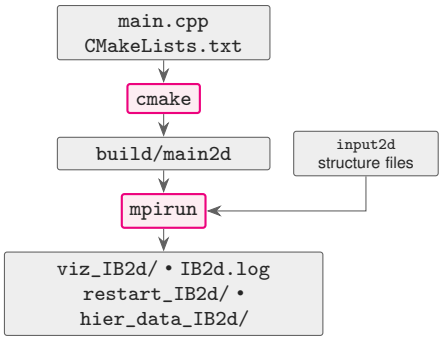
# BUILDING AND RUNNING AN IBAMR APPLICATION

From source files to results, on a laptop or on Puma

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part I • Chapter 3




Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

## LEARNING GOALS

By the end of this chapter you should be able to:

1. build an IBAMR program with CMake, in 2D or 3D;
2. run it with MPI and read what it prints and logs;
3. find your way around an input file and change it safely;
4. run on Puma with a batch script, follow the job, and restart it;
5. estimate how long a run will take and how much disk it needs;
6. port an older course example to current IBAMR;
7. run a parameter sweep as a Slurm job array.

> The examples are the three starter examples of this tutorial: the rubber band (IB/01), the pulsatile channel (IB/02) and the tethered plate (IBFE/01). Every command and message on these slides was run and recorded with IBAMR 0.16.0 and 0.19.0.

# WHAT AN IBAMR APPLICATION IS

```
examples/IB/01-rubber-band/
  CMakeLists.txt           how to build
  main.cpp                 the program
  input2d                  all the settings
  rubber_band_512.vertex   the structure:
  rubber_band_512.spring     points and springs
  generate_mesh2d.m        makes those files
  job.slurm                batch script for Puma
  README.md
```

- IBAMR is a library. Your application is a short C++ program, `main.cpp`, that sets up IBAMR's objects and runs the time-step loop.
- Almost every choice lives in the input file: physical constants, grid, solver settings, output.
- The structure comes from files (`.vertex`, `.spring`, ...) or, for IBFE, a finite element mesh.
- Most course examples share one program and differ in their input and structure files. IB/01 and IB/02 use the same `main.cpp`.

# THE OLD WAY TO BUILD: A MAKEFILE

Until 2025 every course example came with a `Makefile` that pointed at one particular IBAMR install. The 2024 course instructions said to edit these lines:

```
IBAMR_SRC_DIR = /groups/lauram9/ib10/ibamr/IBAMR
IBAMR_BUILD_DIR = /groups/lauram9/ib10/ibamr/ibamr-objs-opt
LIBSNEW = $(LIBS) -lnetcdf -lcurl
...
include $(IBAMR_BUILD_DIR)/config/make.inc
```

With an autoibamr install, `make main2d` stops at once:

```
Makefile:27: /contrib/lauram9/ibamr/IBAMR-0.7.1/config/make.inc: No such file or directory
make: *** No rule to make target '/contrib/lauram9/ibamr/IBAMR-0.7.1/config/make.inc'.  Stop.
```

- autoibamr builds IBAMR with CMake, which creates no `make.inc`.
- IBAMR 0.20 removes Makefile builds of IBAMR altogether.
- So every example in this tutorial now builds with CMake. The Makefiles are kept in `examples/legacy/` and in Part X.

# THE NEW WAY: CMAKELISTS.TXT, LINE BY LINE

```
cmake_minimum_required(VERSION 3.15...3.30)
project(ib01_rubber_band LANGUAGES C CXX)

find_package(IBAMR 0.16.0 REQUIRED)

add_executable(main2d main.cpp)
target_link_libraries(main2d PRIVATE IBAMR::IBAMR2d)
```

| Line | What it does |
|---|---|
| `cmake_minimum_required` | the range turns on newer CMake rules, which also silences two harmless warnings from IBAMR's CMake files |
| `project` | names the project; C and C++ compilers are needed |
| `find_package(IBAMR 0.16.0 ...)` | finds IBAMR 0.16.0 or newer, through `IBAMR_ROOT` from `enable.sh` |
| `add_executable` | the program to build and its source files |
| `target_link_libraries` | links IBAMR and everything it needs; `IBAMR2d` also sets the dimension, `NDIM = 2` |

# CONFIGURE, THEN BUILD

From the example's folder, after loading the install (chapter 2):

```
source ~/autoibamr/configuration/enable.sh      # on Puma: module load gnu13 openmpi5 first
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release # configure: once per build folder
cmake --build build -j 4                        # compile: after every change to main.cpp
```

The last lines you should see (from the rubber band with IBAMR 0.19.0):

```
-- Configuring done (5.6s)
-- Generating done (0.0s)
-- Build files have been written to: /root/work/0.19.0/ib01/build
[ 50%] Building CXX object CMakeFiles/main2d.dir/main.cpp.o
[100%] Linking CXX executable main2d
[100%] Built target main2d
```

- `-S .` is the source folder and `-B build` the build folder. The program ends up as `build/main2d`.
- Changing `input2d` needs no rebuild; changing `main.cpp` does.
- Start over by deleting the `build` folder.

# 3D PROGRAMS, EXTRA SOURCE FILES, DEBUGGING

## A 3D program

```
add_executable(main3d main.cpp)
target_link_libraries(main3d PRIVATE
                      IBAMR::IBAMR3d)
```

The same `main.cpp` can build both: `NDIM` is 2 or 3 depending on the library it links.

## Extra source files

```
add_executable(main2d main.cpp
    update_target_point_positions.cpp)
```

List every `.cpp` file your program needs (IB/03 and IB/04 move target points this way).

## Debugging your own code

Configure a second build folder:

```
cmake -S . -B build-debug \
      -DCMAKE_BUILD_TYPE=Debug
```

This adds debug symbols and turns off optimization for *your* code. To also check IBAMR's internal assertions, build against a debug install of IBAMR (chapter 2, `--enable-debugging`). Debug runs are much slower.

# RUNNING A SIMULATION

```
mpirun -np 4 ./build/main2d input2d                    # a new run on 4 processes
mpirun -np 4 ./build/main2d input2d restart_IB2d 1000  # continue from step 1000
```

- `-np 4` starts 4 MPI processes. IBAMR divides the grid and the structure among them.
- Run from the example's folder: the input file names the structure files without a path, and output folders are made where you start.
- The second form restarts from a saved state, explained later in this chapter.
- On Puma, this line goes *inside* a batch script or an interactive session, never on the login node.

> The rubber band's full run, 20,000 time steps, took 166 s with IBAMR 0.16.0 and 181 s with 0.19.0 on 4 cores of a laptop.

# WHAT THE SCREEN SHOWS

```
IBStandardInitializer:  Reading from input files.
  base filename: rubber_band_512
  assigned to level 3 of the Cartesian grid patch hierarchy

Writing visualization files...

++++++++++++++++++++++++++++++++++++++++++++++++++++++++
At beginning of timestep # 0
Simulation time is 0

At end       of timestep # 0
Simulation time is 0.0001
++++++++++++++++++++++++++++++++++++++++++++++++++++++++
```

- First IBAMR reads the structure files and places the structure on the finest grid level (here level 3).
- It writes the first VisIt frame, at time 0.
- Then it prints two lines at the start and end of every time step.
- The rubber band also prints `integral{F}`, the sum of its forces, every 400 steps; that line comes from the example's own `output_data` function.

# WHAT THE LOG FILE SHOWS: ONE IB TIME STEP

`IB2d.log` (set by `log_file_name`) records each stage of every step. Step 100 of the rubber band:

```
IBHierarchyIntegrator::advanceHierarchy(): time interval = [0.01,0.0101], dt = 0.0001
IBHierarchyIntegrator::preprocessIntegrateHierarchy(): performing Lagrangian forward Euler step
IBHierarchyIntegrator::integrateHierarchy(): computing Lagrangian force
IBHierarchyIntegrator::integrateHierarchy(): spreading Lagrangian force to the Eulerian grid
IBHierarchyIntegrator::integrateHierarchy(): solving the incompressible Navier-Stokes equations
INSStaggeredHierarchyIntegrator::integrateHierarchy(): stokes solve number of iterations = 3
INSStaggeredHierarchyIntegrator::integrateHierarchy(): stokes solve residual norm        = 0.658181
IBHierarchyIntegrator::integrateHierarchy(): interpolating Eulerian velocity to the Lagrangian mesh
IBHierarchyIntegrator::integrateHierarchy(): performing Lagrangian midpoint-rule step
IBHierarchyIntegrator::postprocessIntegrateHierarchy(): CFL number = 0.00987275
IBHierarchyIntegrator::postprocessIntegrateHierarchy(): Eulerian estimate of upper bound on IB point displacement since last regrid
      = 0.450684
```

- The stages are the IB method itself: compute the structure's forces, spread them to the grid, solve the fluid equations, interpolate the velocity back, and move the points.
- Watch the **CFL number** and the **number of Stokes iterations**. A CFL number that keeps growing or iteration counts that climb are early signs of trouble (chapter 16).

# WHERE THE OUTPUT GOES

| Path | What it holds |
|---|---|
| `viz_IB2d/dumps.visit` | list of fluid frames for VisIt: `U`, `P`, `Omega`, ... on the adaptive grid |
| `viz_IB2d/lag_data.visit` | list of structure frames for VisIt (Lagrangian points) |
| `viz_IB2d/visit_dump.NNNNN/` | one fluid frame per folder; `NNNNN` is the time step |
| `viz_IB2d/lag_data.cycle_NNNNNN/` | one structure frame per folder |
| `viz_IB2d/output.ex2` | IBFE only: the finite element mesh and its data, all frames in one file |
| `hier_data_IB2d/` | data written by the example's `output_data` (rubber band: positions `X.NNNNN` and forces `F.NNNNN`) |
| `restart_IB2d/restore.NNNNNN/` | saved states for restarting |
| `IB2d.log` | the log file |

Sizes from the full runs: rubber band 39 MB of VisIt files for 51 frames plus 1.4 MB of data files; tethered plate 14 MB for 41 frames.

# THE INPUT FILE: VARIABLES AND BLOCKS

```
// physical parameters
L = 1.0                 // domain length (m)
MU = 1.0e2              // viscosity (N s/m^2)

// grid spacing parameters
MAX_LEVELS = 4
REF_RATIO  = 4
N = 8
NFINEST = (REF_RATIO^(MAX_LEVELS - 1))*N
DX = (1.0*L)/NFINEST

IBMethod {
   delta_fcn       = DELTA_FUNCTION
   enable_logging = ENABLE_LOGGING
}
```

- **Variables** at the top: `NAME = value`. Values can be numbers, `"strings"`, `TRUE`/`FALSE`, or expressions using `+ - * / ^` and functions such as `int()`.
- **Blocks**: `Name { key = value }`. Each block configures one IBAMR object; `main.cpp` hands it over by name, for example `getComponentDatabase("IBMethod")`.
- `//` starts a comment.
- A misspelled key is ignored without a warning, and the object quietly uses its default. With `viz_dump_intervall` for `viz_dump_interval`, the rubber band ran normally and wrote no VisIt files at all.

# THE BLOCKS IN A TYPICAL IB INPUT FILE

| Block | What it sets |
|---|---|
| `CartesianGeometry` | domain size and coarse grid (`domain_boxes`, `x_lo`, `x_up`), periodic directions |
| `GriddingAlgorithm` | number of levels, refinement ratio, patch sizes |
| `INSStaggeredHierarchyIntegrator` | fluid: `mu`, `rho`, CFL limit, which fields to output |
| `IBHierarchyIntegrator` | start and end time, maximum time step, what to do when it must change |
| `IBMethod` or `IBFEMethod` | delta function and, for IBFE, quadrature and coupling choices |
| `IBStandardInitializer` | which structure files to read and on which grid level |
| `VelocityBcCoefs_0`, `_1` | boundary conditions for each velocity component (chapter 7) |
| `VelocityInitialConditions` | starting velocity |
| `Main` | log file, output types and intervals, restart files |
| `StandardTagAndInitialize`, `LoadBalancer`, `TimerManager` | where to refine, how to divide work, timing reports |

# THE MAIN BLOCK

```
Main {
   solver_type = SOLVER_TYPE
   log_file_name                = "IB2d.log"
   log_all_nodes                = FALSE
   viz_writer                   = "VisIt","Silo"
   viz_dump_interval            = VIZ_INTERVAL
   viz_dump_dirname             = "viz_IB2d"
   visit_number_procs_per_file = 1
   restart_dump_interval        = 0
   restart_dump_dirname         = "restart_IB2d"
   data_dump_interval           = DATA_INTERVAL
   data_dump_dirname            = "hier_data_IB2d"
   timer_dump_interval          = 0
}
```

- Every `*_interval` counts time steps, not seconds. `0` turns that output off.
- `viz_writer`: `"VisIt"` for the fluid, `"Silo"` for IB structures, `"ExodusII"` for IBFE meshes.
- `data_dump_interval` calls the program's own `output_data`, if it has one.
- Set `restart_dump_interval` for any run that might not finish in one job.

## TIME STEP AND RUN LENGTH

The number of steps is `END_TIME / DT`. For the rubber band (`END_TIME` = 2 s):

| `DT` | steps | time on 4 cores | result |
|---|---|---|---|
| $1.0 \times 10^{-6}$ (2020 version) | 2,000,000 | about 6 hours (estimated) | reference |
| $1.0 \times 10^{-4}$ (this version) | 20,000 | 166–181 s | same band shape to 4 digits |
| $5.0 \times 10^{-4}$ | 4,000 | | still the same shape |
| $1.0 \times 10^{-3}$ | | | stops after 24 steps: `Time step size change encountered` |

- The largest stable `DT` depends on the structure's stiffness, the grid and the flow. This band is very viscous, which is why such a large step works.
- Find a good `DT` the same way: run a short time at several values and compare a number you care about.
- `CFL_MAX` caps how far fluid can move in one step, in grid cells. When the flow is fast, the CFL limit can become smaller than `DT`.

# “TIME STEP SIZE CHANGE ENCOUNTERED”

```
P=00000:ERROR MESSAGE:
P=00000:IBHierarchyIntegrator::preprocessIntegrateHierarchy():  Time step size change encountered.
P=00000:Aborting.
```

- Each step IBAMR uses the smaller of `DT` and the step allowed by the CFL limit. With `ERROR_ON_DT_CHANGE = TRUE`, a change is an error, so the run stops as soon as the flow gets fast enough that the CFL limit falls below `DT`.
- **If the flow really is that fast**, lower `DT`.
- **If it should not be**, look for the cause first. In IBFE/01 with a very soft plate (`SHEAR_MODULUS = 1`), the free half crumples, stirs up fast flow, and the run stops near t = 1.6. Halving `DT` only moved the stop to t = 2.05.
- `ERROR_ON_DT_CHANGE = FALSE` lets IBAMR shrink the step instead of stopping. Output intervals then no longer fall at evenly spaced times, because they count steps.

# OUTPUT INTERVALS THAT SCALE WITH THE RUN

```
DT            = 1.0e-4
END_TIME      = 2.0
VIZ_INTERVAL  = int(END_TIME/(50*DT))   // 50 VisIt frames, whatever DT and END_TIME are
DATA_INTERVAL = int(END_TIME/(50*DT))
...
   viz_dump_interval  = VIZ_INTERVAL
   data_dump_interval = DATA_INTERVAL
```

- The course versions set fixed step counts. When `DT` changed, the output went wrong: the pulsatile channel wrote a VisIt frame every 10,000 steps, only 3 frames in all, and the 2020 rubber band's data interval would have written 1,000 pairs of position and force files.
- IBAMR's own examples use the same `int(END_TIME/(n*DT))` pattern.

# THE MOST COMMON FIRST ERROR: WINDOWS LINE ENDINGS

An input file saved on Windows ends each line with an extra invisible character. IBAMR stops before the first time step:

```
Warning in input2d at line 2 column 10 : Illegal character token in input
Error in input2d at line 2 column 10 : syntax error
P=00000:Program abort called in file
     ``../../../../SAMRAI-2.4.4/source/toolbox/inputdb/InputManager.C'' at line 172
P=00000:ERROR MESSAGE:
P=00000:InputManager exiting...
```

Fix the file on the cluster:

```
dos2unix input2d
# or, if dos2unix is missing:
sed -i 's/\r$//' input2d
```

- Or save with Unix (LF) endings: Notepad++ *Edit* → *EOL Conversion*; VS Code, click `CRLF` in the status bar.
- Older course files ended every line with `//`, which hid the extra character inside a comment.

Recorded in a 2024 course run on Puma (file IBAMR.out, kept in the course repository).

# TWO MORE INPUT ERRORS YOU WILL MEET

**A stray character** gives the same kind of message. Two course input files ended with `\\`:

```
Warning in input2d at line 219 column 2 : Illegal character token in input
Error in input2d at line 219 column 2 : syntax error
```

**An expression muParser cannot read** stops the run at the first time step. muParser has no `if()`:

```
P=00001:muParserRobinBcCoefs::setDataOnPatch():
P=00001:  error: Unexpected token "if" found at position 0.
P=00001:  in:    if( X_1 <= -0.5*d,0.0,if( X_1 >= 0.5*d,0.0,4.0/(d*d)*(0.5 + ...
```

Write `condition ? a : b` instead (one line in the file):

```
"(X_1 <= -0.5*d) ? 0.0 : ((X_1 >= 0.5*d) ? 0.0 : 4.0/(d*d)*(0.5 + 0.5*sin(2*PI*f*t))*((0.25*d*d) - X_1*X_1))"
```

In these expressions `X_0` and `X_1` are the coordinates and `t` is time. Chapter 7 covers boundary conditions.

# ON PUMA: NEVER ON THE LOGIN NODE

- When you log in you land on a **login node**, shared by everyone. Edit files and submit jobs there, but do not run simulations or builds there.
- The compiler and MPI modules are available only on **compute nodes**.
- For short interactive work, ask for a compute node:

```
interactive                  # one core for one hour
module load gnu13 openmpi5
module list                  # what is loaded; module avail lists everything
```

- For real runs, write a **batch script** and submit it to Slurm, the scheduler. Slurm starts the job on compute nodes when they are free.

Adapted from the course guide “Running IBAMR/IBFE on super computers” (Nick Battista, 2014; University of Arizona version 2024).

# THE BATCH SCRIPT, LINE BY LINE

```
#!/bin/bash
#SBATCH --job-name=ib01-rubber-band
#SBATCH --account=lauram9
#SBATCH --partition=standard
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --mem-per-cpu=4gb
#SBATCH --time=00:30:00
#SBATCH --output=%x-%j.out

module load gnu13 openmpi5
source ~/autoibamr/configuration/enable.sh

cd "$SLURM_SUBMIT_DIR"
mpirun -np "$SLURM_NTASKS" ./build/main2d input2d
```

- `--account`: your PI's group.
- `--ntasks`: MPI processes; `$SLURM_NTASKS` repeats it for `mpirun`.
- `--time`: the job is stopped when this runs out. Hours:minutes:seconds.
- `--output`: `%x` is the job name and `%j` the job number.
- Each `#SBATCH` option starts with *two* hyphens.
- `$SLURM_SUBMIT_DIR` is the folder you submitted from.
- Every example has a `job.slurm` like this one.

# SUBMITTING AND MANAGING JOBS

```
sbatch job.slurm          # submit; prints "Submitted batch job <number>"
squeue -u $USER           # your jobs: PD = waiting, R = running
scancel <number>          # stop one job
scancel -u $USER          # stop all of your jobs
```

Two shortcuts from the course notes, for your `~/.bashrc`:

```
alias showqueue='squeue -u $USER'
alias deletealljobs='scancel -u $USER'
```

> Keep a record of every job you submit: its number, the date, what it simulates, and the key settings (Reynolds number, `DT`, output interval). You will need it when you analyze the results months later.

# FOLLOWING A RUNNING JOB

From the job's folder:

```
tail -f IB2d.log                              # watch the log grow; Ctrl-C to stop watching
grep "At end" ib01-rubber-band-*.out | tail -1   # the last finished step
grep "CFL number" IB2d.log | tail -3          # is the flow speeding up?
ls viz_IB2d | tail -3                         # the newest frames
```

- Divide the step reached by the total number of steps to see how far along the run is.
- The `.out` file holds the screen output and any error messages; the log has the detail.
- To read long files, `less` is quick: space for the next page, `G` for the end, `q` to quit. Chapter 1 covers `nano` and `vi`.

# HOW MANY CORES?

- More processes help only if each one still has plenty of work. Small 2D problems stop speeding up quickly, because the processes spend their time exchanging data.
- Measured for the rubber band (2D, 512 cells across on the finest level): **10.5 ms per step on 4 processes and 10.2 ms on 8**. Doubling the cores bought nothing.
- 3D runs have far more cells and do benefit from many cores.
- The course notes add: if a run is unexpectedly slow, try fewer processes, use a power of 2, and check that `DT` is not much smaller than it needs to be.

> Test before a long job: run a few hundred steps on 2, 4 and 8 processes, compare the time per step, and pick the smallest count that is nearly as fast.

# WHEN A JOB STOPS EARLY

| What you see | What to do |
|---|---|
| the `.out` file reports a cancellation *due to time limit* | raise `--time`, or restart from the last restart file |
| the job was killed for memory, or a PETSc error appears at the end | some PETSc errors mean the job ran out of memory: raise `--mem-per-cpu` or use more nodes |
| `Time step size change encountered` | the flow became faster than `DT` allows; see slide 16 |
| `Illegal character token in input` | line endings or a stray character; see slides 18–19 |
| the job waits in the queue for a long time | check the reason with `squeue -u $USER`; ask for fewer cores or less time |

Always read the end of both `IB2d.log` and the `.out` file first: the last lines usually name the problem.

Partly adapted from the course's "Debugging tips" slides (2019–2020).

# RESTARTING A RUN

In `Main`, save the state every 500 steps:

```
restart_dump_interval = 500
restart_dump_dirname  = "restart_IB2d"
```

Each save is a folder:

```
restart_IB2d/restore.000000
restart_IB2d/restore.000500
restart_IB2d/restore.001000  ...
```

Continue from step 1000:

```
mpirun -np 4 ./build/main2d input2d \
       restart_IB2d 1000
```

- Tested with the rubber band: a run to t = 0.2 saved 5 restart folders; the restart began at step 1000 and ran to t = 0.3 (step 2999).
- You may raise `END_TIME` in `input2d` before restarting.
- Restart folders are large. Keep the last one or two.
- IBAMR 0.19 adds `RestartCleaner`, a class a program can call to keep only the newest folders.

Restart with the same number of processes as the original run. With 2 instead of 4, the restart stops with `Unable to open HDF5 file restart_IB2d/restore.001000/nodes.00002/proc.00000`.

# AFTER A RESTART, REBUILD VISIT'S FRAME LISTS

A restarted run rewrites `dumps.visit` and `lag_data.visit` starting from the restart step. In the test, the frame folders for the whole run were on disk (77 of each), but `dumps.visit` listed only the 35 written after the restart. Rebuild both lists from `viz_IB2d/`:

```
cd viz_IB2d
ls -d visit_dump.* | sort -t. -k2 -n | sed 's#$#/summary.samrai#' > dumps.visit
for d in $(ls -d lag_data.cycle_* | sort -t_ -k3 -n); do
    echo "$d/$d.summary.silo"
done > lag_data.visit
```

- Afterwards each list had all 77 frames, and every listed file existed.
- Frames between the restart step and the old end are overwritten by the restarted run.
- IBFE: `output.ex2` is appended to rather than rewritten, because the program calls `exodus_io->append(...)` on restart. IBFE/01 does this, and it also saves the plate's finite element data with each restart folder.

# HOW LONG WILL IT TAKE, AND HOW MUCH DISK?

| Example | steps | time on 4 cores | VisIt output |
|---|---:|---:|---|
| IB/01 rubber band (2D, 4 levels) | 20,000 | 166 s | 39 MB, 51 frames |
| IB/02 pulsatile channel (2D, 3 levels) | 20,000 | 318 s | 49 MB, 51 frames |
| IBFE/01 tethered plate (2D, 2 levels) | 4,000 | 26 s | 14 MB, 41 frames |

- Time $\approx$ (time per step) $\times$ (number of steps). Measure the time per step from a short run, then multiply.
- Disk $\approx$ (size of one frame) $\times$ (number of frames). 3D frames are much larger: plan the number of frames before a long run.
- Laptop timings, IBAMR 0.16.0. Puma's cores are of a similar speed, but runs also wait in the queue.

# PORTING AN OLDER EXAMPLE: WHAT BREAKS

Compiling unmodified course programs against IBAMR 0.19.0 with the new `CMakeLists.txt`:

```
main.cpp:31:10: fatal error: IBAMR_config.h: No such file or directory
```

Every course program stops here first. After deleting the two old configuration includes, the IB programs build. The UNC IBFE programs then stop at

```
main.cpp:368:9: error: 'AutoPtr' was not declared in this scope
```

and the two-plate IBFE program also at

```
error: no matching function for call to 'IBAMR::IBFEMethod::IBFEMethod(const char [11],
       SAMRAI::tbox::Pointer<SAMRAI::tbox::Database>, std::vector<libMesh::Mesh*>&, int)'
```

- `IBAMR_config.h` and `IBTK_config.h` existed only in Makefile builds of IBAMR.
- `AutoPtr` was removed from libMesh; use `std::unique_ptr`.
- Several meshes are now passed as `std::vector<MeshBase*>`.

# PORTING CHECKLIST

1. Add a `CMakeLists.txt` (copy one from an example) and set aside the `Makefile`.
2. Delete `#include <IBAMR_config.h>` and `#include <IBTK_config.h>`.
3. Start IBAMR with `IBTKInit` (next slide). The old start-up still compiles, but `IBTKInit` also raises a SAMRAI limit on the number of data fields, which runs with many variables can exceed.
4. IBFE: `AutoPtr` → `std::unique_ptr`; `std::vector<Mesh*>` → `std::vector<MeshBase*>`; `libMeshEnums::Order` → `libMesh::Order`; `IBFEMethod::FORCE_SYSTEM_NAME` → `getForceSystemName()`.
5. Optional tidying: `MathUtilities<double>::equalEps` → `IBTK::rel_equal_eps`; `SAMRAI_MPI` → `IBTK_MPI`; `NULL` → `nullptr`.
6. Input files: Unix line endings, no stray characters, `if()` rewritten as `? :`, obsolete keys (`use_IB_spread_operator`, `use_IB_interp_operator`) removed.
7. Check `DT`, `END_TIME` and the output intervals; make the intervals scale with the run.
8. Write a `job.slurm` for Puma; delete scripts for retired clusters.
9. Build against 0.16.0 and 0.19.0 if you can, run a few hundred steps, and look at the output.

# BEFORE AND AFTER: STARTING IBAMR

## Course versions (before 2021)

```
#include <IBAMR_config.h>
#include <IBTK_config.h>
...
int main(int argc, char* argv[])
{
    PetscInitialize(&argc, &argv, NULL, NULL);
    SAMRAI_MPI::setCommunicator(PETSC_COMM_WORLD);
    SAMRAI_MPI::setCallAbortInSerialInsteadOfExit();
    SAMRAIManager::startup();
    {
        ...
    }
    SAMRAIManager::shutdown();
    PetscFinalize();
    return 0;
}
```

## Now (IBAMR 0.8 and later)

```
#include <ibtk/IBTKInit.h>
...
int main(int argc, char* argv[])
{
    // starts MPI, PETSc, libMesh and SAMRAI,
    // and stops them when main() ends
    IBTKInit ibtk_init(argc, argv, MPI_COMM_WORLD);
    // IBFE programs also need:
    const LibMeshInit& init =
        ibtk_init.getLibMeshInit();
    {
        ...
    }
    return 0;
}
```

# BEFORE AND AFTER: IBFE FUNCTION ARGUMENTS

Stress and force functions take different arguments than in course code written before 2018. The current form, used by every IBFE example in this tutorial:

```
void PK1_dev_stress_function(TensorValue<double>& PP,          // output: the stress
                             const TensorValue<double>& FF,     // deformation gradient
                             const libMesh::Point& X,           // current position
                             const libMesh::Point& s,           // reference position
                             Elem* const elem,
                             const std::vector<const std::vector<double>*>& var_data,
                             const std::vector<const std::vector<VectorValue<double> >*>& grad_var_data,
                             double time,
                             void* ctx)
```

- Older code passed `const vector<NumericVector<double>*>& system_data` instead of `var_data` and `grad_var_data`. Replace that one argument by these two.
- Force functions (`LagBodyForceFcnData`) take the same arguments with `VectorValue<double>& F` first.
- Mark unused arguments as `/*elem*/` to avoid compiler warnings.

The argument changes were first collected in the course slides “Run your IBAMR simulation on Dogwood” (2018–2020).

# RUNNING MANY SIMULATIONS

**Goal:** run the same model for many parameter values, such as Reynolds or Womersley number, frequency or stiffness.

1. Make a table of parameter values, one run per line.
2. For each line, make a folder with its own input file.
3. Run each folder as a job, and keep each run's output in its folder.
4. Check the generated input files *before* submitting anything.

> The 2018 course version did steps 1–3 with a shell script that edited the input template by *line number* and submitted one LSF job per run. The next slide shows why editing by name is safer. The full 2018 scripts are in Part X.

# THE 2018 APPROACH, AND WHAT WENT WRONG

```
cut -f 3 allpara.txt > Freq.txt                  # one column of the table
numlines=$(grep -c "^" allpara.txt)              # number of runs
WO=0
for i in `seq 1 $numlines`; do
  awk -v var="$WO" 'NR==7 {$0="WO = "'"var"'"   // Womersley number"} 1' template-input2d > input2dw${i}
  Freq=$(awk -v var="$i" 'NR==var' Freq.txt)
  awk -v var="$Freq" 'NR==5 {$0="FRE = "'"var"'"   // Pumping Frequency (Hz)"} 1' input2dw${i} > input2dw${i}f${i}
  ...
done
```

- `WO` is never read from the table, so every input file gets `WO = 0`. The template computes `MU = 2.0*PI*FRE*RHO*D*D/(4.0*WO*WO)`, which divides by zero.
- Line numbers (`NR==7`, `NR==201`) break silently as soon as a line is added to the template.
- The submit script read `input2d-files/` but made a folder called `parameters-files/`.

# A STURDIER APPROACH: REPLACE VALUES BY NAME

```
# copy input2d into run_3, replacing the MU line wherever it is
awk -v mu="1" '/^MU[ \t]*=/ {print "MU = " mu; next} {print}' input2d > run_3/input2d
grep "^MU" run_3/input2d          # check: MU = 1
```

- The pattern `/^MU[ \t]*=/` finds the line that starts with `MU`, so it keeps working when lines move.
- Put every parameter you sweep as a variable at the top of the input file, and let the blocks use it (`mu = MU`).
- Derived quantities belong in the input file too, so each run's file is complete. For a Womersley sweep:

```
WO  = 5.05                                  // set by the sweep
MU  = 2.0*PI*FRE*RHO*D*D/(4.0*WO*WO)        // viscosity that gives this Womersley number
```

# SLURM JOB ARRAYS: ONE SCRIPT, MANY RUNS

```
#SBATCH --array=1-3
#SBATCH --output=sweep-%A_%a.out
...
cd "$SLURM_SUBMIT_DIR"
# line i of params.txt is the MU for task i
MU=$(sed -n "${SLURM_ARRAY_TASK_ID}p" params.txt)
RUN=run_${SLURM_ARRAY_TASK_ID}
mkdir -p "$RUN"
cp rubber_band_512.vertex rubber_band_512.spring "$RUN"/
awk -v mu="$MU" '/^MU[ \t]*=/ {print "MU = " mu; next} {print}' \
    input2d > "$RUN"/input2d
cd "$RUN"
mpirun -np "$SLURM_NTASKS" ../build/main2d input2d
```

- `--array=1-3` starts 3 copies of the job; each sees its own `$SLURM_ARRAY_TASK_ID`.
- `%A` is the array's job number and `%a` the task number.
- The complete script is `job_array.slurm` in IB/01, with `params.txt` holding 100, 10 and 1.
- Tested by running the three tasks locally with the Slurm variables set by hand: each `run_i` got its own `MU` and finished.
- `scancel <array job number>` stops all tasks.

# GETTING RESULTS BACK TO YOUR COMPUTER

- Copy finished runs off the cluster soon after they end, and keep them on an external drive: one run can take from hundreds of megabytes to many gigabytes.
- Graphical: FileZilla with SFTP to `filexfer.hpc.arizona.edu` (setup in chapter 1).
- Command line, from your own computer:

```
scp -r netid@filexfer.hpc.arizona.edu:/xdisk/<pi>/<netid>/run_3 .
rsync -av netid@filexfer.hpc.arizona.edu:/xdisk/<pi>/<netid>/run_3 .   # resumes if interrupted
```

- To move only what you need for VisIt, copy `viz_IB2d/` and leave the restart folders behind.

Data-handling advice adapted from the course guide “Running IBAMR/IBFE on super computers” (Nick Battista, 2014).

# SUMMARY

1. Build with CMake: `cmake -S . -B build`, then `cmake --build build`. Makefile builds do not work with autoibamr.
2. Run with `mpirun -np N ./build/main2d input2d`; read the screen, `IB2d.log`, and `viz_IB2d/`.
3. Input files: variables at the top, one block per object, Unix line endings, `? :` instead of `if()`. Misspelled keys are ignored silently.
4. `DT` and `END_TIME` set the cost; make output intervals scale with them.
5. On Puma, run in batch jobs: `sbatch`, `squeue`, `scancel`. Save restart files for long runs and rebuild the VisIt lists after a restart.
6. Porting: CMake, no old configuration headers, `IBTKInit`, and the IBFE changes.
7. Sweeps: parameters as named variables, one folder per run, a Slurm job array.

Next: Part II introduces IB2d; Part III goes through IBAMR's structure files, boundary conditions and parameters.

# REFERENCES AND CREDITS


- IBAMR developers. *Building IBAMR applications*. https://ibamr.github.io/applications.
- IBAMR source, `examples/IB/explicit/ex1` and `examples/IBFE/explicit/ex2` (versions 0.16.0 and 0.19.0): the programs the course examples are based on.
- SchedMD. *Slurm job arrays*. https://slurm.schedmd.com/job_array.html.
- University of Arizona HPC documentation. https://hpcdocs.hpc.arizona.edu.
- muParser expression parser. https://beltoforion.de/en/muparser.
- Course material reused here: “Running IBAMR/IBFE on super computers” (Nick Battista, 2014; UA version 2024), “Run your IBAMR simulation on Dogwood” and “Debugging tips” (2018–2020), “Running many simulations, part 2” (2018), “Updating examples” (2024).